

Il linguaggio PHP

Lorenzo Marconi

Tecnologie e Sistemi Web

Corso di laurea in Ingegneria Informatica e Automatica

Facoltà di Ingegneria dell'Informazione, Informatica e Statistica



SAPIENZA
UNIVERSITÀ DI ROMA

Anno accademico 2025/2026

- 1 Introduzione
- 2 Note pratiche
- 3 Il linguaggio PHP
- 4 PHP come linguaggio lato server
- 5 Connessione a un database tramite PHP

PHP (acronimo ricorsivo che sta per *PHP Hypertext Preprocessor*) è un linguaggio di scripting lato server.

- Preprocessore di ipertesti open source. In particolare, è un linguaggio di scripting **interpretato**, usato principalmente per la programmazione di pagine web dinamiche
- Nel corso delle esercitazioni useremo la versione PHP 8
- Per farne il download:
 - su sistemi Windows si consiglia l'installazione di **XAMPP** (cross-platform, quindi disponibile anche per altri SO)
 - su sistemi Linux è sufficiente lanciare, tramite shell, il seguente comando: `sudo apt install php` (altri pacchetti utili: `libapache2-mod-php php-pgsql`)

- È un linguaggio di scripting, general-purpose e open source largamente utilizzato
- Semplice da imparare. La sua sintassi è simile a C, C++ e Java
- Particolarmente indicato per lo sviluppo Web. Infatti è un linguaggio **HTML-embedded**, ovvero permette di inserire codice interpretato dinamicamente all'interno di una pagina HTML
- Si integra nativamente con i più diffusi DBMS tra cui gli open source PostgreSQL e MySQL
- È supportato dai più diffusi web server, fra cui Apache e nginx

Pagine statiche e dinamiche

Consideriamo la classica architettura client-server

Il codice PHP viene usato per generare **dinamicamente** i documenti HTML che il client deve ricevere e visualizzare nel browser.

- In un sito web statico, il documento HTML (non-PHP) viene preso dal file system del web server e inviato direttamente al client
- In un sito web dinamico, i file del web server (ad esempio, file PHP) vengono prima passati all'interprete PHP del web server stesso, e successivamente viene generato il documento HTML (o dati di altro tipo) da inviare al client



Di seguito un semplice file con estensione .php:

```
<html>
  <head>
    <title>Primo script PHP</title>
  </head>
  <body>
    <?php
      echo "<h1>Ciao. Sono un semplice script PHP</h1><br/>";
      echo "<h2>Vengo convertito in codice HTML dall'interprete PHP</h2>";
    ?>
  </body>
</html>
```

Questo è il corrispondente output HTML generato dall'interprete PHP:

```
<html>
  <head>
    <title>Primo script PHP</title>
  </head>
  <body>
    <h1>Ciao. Sono un semplice script PHP</h1><br/>
    <h2>Vengo convertito in codice HTML dall'interprete PHP</h2>
  </body>
</html>
```

echo è uno dei costrutti basilari di PHP. Esso permette infatti di stampare un valore sotto forma di stringa

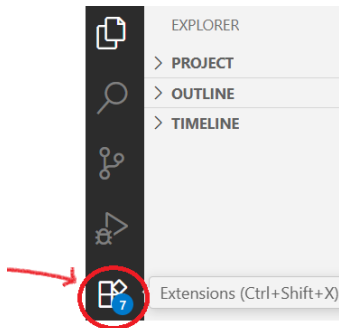
Da tenere bene a mente:

- ❶ Un file PHP può essere composto da vari script PHP mescolati a codice lato client (ad esempio, HTML e JavaScript)
- ❷ Ogni script PHP è compreso tra un apposito tag di apertura “<?php” ed un apposito tag di chiusura “?>”. L'interprete PHP esegue solo codice contenuto all'interno di questi due delimitatori
- ❸ Spesso, l'**output** desiderato è una risorsa (es. un documento HTML) da consegnare al client
- ❹ Il codice PHP stesso **non** viene consegnato al client

- 1 Introduzione
- 2 Note pratiche**
- 3 Il linguaggio PHP
- 4 PHP come linguaggio lato server
- 5 Connessione a un database tramite PHP

Lanciare server PHP da Visual Studio Code - Parte 1

Apriamo Visual Studio Code ed andiamo ad installare, tramite la barra delle attività (si veda immagine di sotto), l'estensione *PHP Server*.

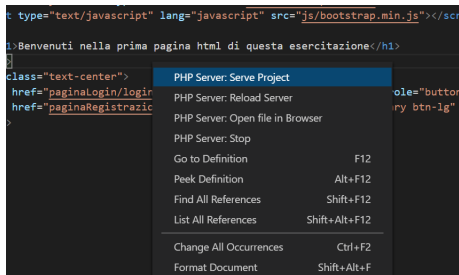


Per settare PHP su VS Code, seguire questa procedura:

- 1 Cercare il percorso dell'eseguibile di PHP.
 - Per chi usa XAMPP, il path di default dovrebbe essere `"C:\\xampp\\php\\php.exe"`.
 - Per chi ha installato PHP su un sistema Unix-like, l'eseguibile dovrebbe trovarsi sotto `/usr/bin/php` (assicurarsi che questo sia il path restituito dal comando `which php`).
- 2 Su VS Code, andare su: File → Preferenze → Impostazioni.
Da qui, sotto il tab Utente, andare su Estensioni → PHP → PHP Validate: Executable Path e cliccare su "Modifica in settings.json".
Nel file che si aprirà, associare alla chiave `php.validate.executablePath` il percorso dell'eseguibile di PHP, e infine salvare.
- 3 Sempre sotto il tab Utente, andare su Estensioni → PHP Server configuration → PHP Path e scrivere nella textbox nuovamente il percorso dell'eseguibile di PHP.

Come eseguire un progetto

Per eseguire un qualsiasi progetto che utilizzi PHP tramite editor di Visual Studio Code, recarsi nella pagina principale del progetto (ad esempio, `index.php`), dopodiché cliccare sul tasto destro e, infine, su *PHP Server: Serve Project* (si veda immagine di sotto):



- 1 Introduzione
- 2 Note pratiche
- 3 Il linguaggio PHP**
- 4 PHP come linguaggio lato server
- 5 Connessione a un database tramite PHP

- PHP è **case sensitive** (distingue tra maiuscole e minuscole)
- Non esiste un costrutto di dichiarazione di **variabili**
- Il nome delle variabili è preceduto dal simbolo \$. Ad es., “\$a=5;” è una istruzione PHP che assegna alla variabile di nome a il valore 5
- Si può verificare se una variabile è stata inizializzata tramite la chiamata alla funzione `isset($var)` che ritorna `true` se e solo se la variabile `var` è stata precedentemente inizializzata
- Supporta **commenti** multilinea con la stessa sintassi di C++ e Java: `/* Questo è un commento in PHP */`

In PHP le variabili possono contenere valori di tipo diverso senza dichiarazione esplicita.

- **int**: numeri interi, ad esempio 42
- **float**: numeri reali, ad esempio 3.14
- **string**: sequenze di caratteri, ad esempio "ciao"
- **bool**: valori logici true e false
- **null**: assenza di valore
- **array**: collezioni di valori
- **object**: istanze di classi

La tipizzazione in PHP è dinamica: il tipo di una variabile dipende dal valore che contiene

PHP effettua spesso conversioni automatiche (*casting*) tra tipi diversi

- (int) "10" restituisce 10
- (float) "2.5" restituisce 2.5
- (string) 100 restituisce "100"
- `gettype($x)` restituisce il tipo di `$x`
- `is_int($x)`, `is_string($x)`, `is_array($x)` controllano il tipo

Esempio

```
$a = "12";  
$b = 5;  
$c = $a + $b;    // 17
```

- Addizione: $\$a + \b
- Sottrazione: $\$a - \b
- Moltiplicazione: $\$a * \b
- Divisione: $\$a / \b
- Modulo: $\$a \% \b

```
$x = 12;
```

```
$y = 5;
```

```
$x + $y;      /* 17 */
```

```
$x - $y;      /* 7 */
```

```
$x * $y;      /* 60 */
```

```
$x / $y;      /* 2.4 */
```

```
$x % $y;      /* 2 */
```


- AND: `$a && $b`
- OR: `$a || $b`
- NOT: `!$a`

```
$a = true;  
$b = false;
```

```
$a && $b;      /* false */  
$a || $b;      /* true  */  
!$a;           /* false */
```

Operatori di confronto

- Uguale a: `==`
- Uguale a e dello stesso tipo: `===`
- Non uguale a: `!=`
- Non uguale e di tipo diverso di: `!==`
- Minore di: `<`
- Maggiore di: `>`

```
$x = 10;  
$y = "10";
```

```
$x == $y;      /* true */  
$x === $y;    /* false */  
$x != $y;      /* false */  
$x !== $y;     /* true */  
$x < $y;       /* false */  
$x > $y;       /* false */
```

- Selezione:

- `if (condizione1) { ... }`
 `elseif (condizione2) { ... }`
 `...`
 `else { ... }`

- `switch ($var) {`
 `case "a": ... break;`
 `case "b": ... break;`
 `...`
 `default: ... }`

- Iterazione:

- `while (condizione) { ... }`
- `do { ... } while (condizione)`
- `for ($i=0; $i<$var; $i++) { ... }`
- costrutto `foreach`¹
- keyword `break` e `continue`

¹cfr. slide 24

- PHP 8 introduce il costrutto `match` in alternativa a `switch`

Esempio

```
$valore = 2;  
  
$testo = match ($valore) {  
    1 => "uno",  
    2 => "due",  
    3 => "tre",  
    default => "altro"  
};
```

- `match` restituisce un valore
- Ogni ramo è un'espressione
- A differenza di `switch`, il confronto è stretto (`===`)
- Non serve `break`

In PHP le **stringhe** possono essere espresse in due maniere:

- 1 Delimitata da apici singoli `$a = 'World';`
- 2 Delimitata da doppi apici `$a = "World";`

Quando si usano gli **apici**, la stringa viene interpretata esattamente come è scritta. Quando si usano le **virgolette**, le eventuali variabili contenute nella stringa vengono sostituite con il loro valore

```
$b = 'Hello $a'; /* Il valore della variabile $b dopo  
questa istruzione sarà la stringa "Hello $a" */
```

```
$c = "Hello $a"; /* Il valore della variabile $c dopo  
questa istruzione sarà la stringa "Hello World" */
```

L'operazione di **concatenazione** di stringhe è il punto:

- `$a = "Hello";`
- `$b = "World";`
- `$c = "$a" . " " . "World"; /* Il valore della variabile $c dopo questa istruzione sarà la stringa "Hello World" */`

Altre operazioni utili su stringhe:

- **strlen**(\$c) restituisce la lunghezza della stringa \$c (11, nel caso di sopra)
- **substr**(\$c,3,4) restituisce la stringa dal terzo carattere di \$c per i prossimi 4 caratteri (lo W, nel caso di sopra)
- **strpos**(\$c,\$b) restituisce la posizione della prima occorrenza della stringa \$b all'interno di \$c, oppure false nel caso non venga trovata alcuna occorrenza (6, nel caso di sopra)
- **strrev**(\$c) restituisce la stringa inversa di \$c (dlrow olleH, in questo caso)

PHP gestisce un unico tipo di **array**, le cui chiavi possono essere numeriche o associative.

Esempio di array con chiavi **associative**:

- `$a = array("foo" => "pippo", "bar" => "pluto");`
- `$a["foo"]` è la stringa "pippo"

Esempio di array con chiavi **numeriche**:

- `$a = array("pippo", "pluto");`
- `$a[0]` è la stringa "pippo"

Esempio di array con chiavi **"miste"**:

- `$a = array("foo" => "pippo", "pluto");`
- `$a[0]` è la stringa "pluto"

Sintassi più moderna: `[...]` anziché `array(...)`

Per scorrere gli elementi di un array `$a` in PHP si può usare il seguente costrutto:

```
foreach ($arr as $index => $var) {  
    ...  
}
```

che scorre l'array `$arr` associando alla variabile `$var` i valori contenuti, e alla variabile `$index` le chiavi (numeriche o associative a seconda dell'array `$arr`) corrispondenti

Se non si specifica una variabile per la chiave, si itera solo sui valori

Di seguito un file con estensione .php:

```
<html>
  <head>
    <title>Script Array PHP</title>
  </head>
  <body>
    <?php
      $a = array('foo' => 'pippo', 'bar' => 'pluto', 'altro' => 'paperino');
      foreach ($a as $index => $var){
        echo "Valore associato all'indice " . $index . ": " . $var . "<br/>";
      }
    ?>
  </body>
</html>
```

Questo è il corrispondente output HTML generato dall'interprete PHP:

```
<html>
  <head>
    <title>Script Array PHP</title>
  </head>
  <body>
    Valore associato all'indice foo: pippo<br/>
    Valore associato all'indice bar: pluto<br/>
    Valore associato all'indice altro: paperino<br/>
  </body>
</html>
```

Gli array dispongono di molte funzioni built-in. Ad esempio:

- `count($a)`: numero di elementi dell'array
- `in_array($x, $a)`: verifica se un valore è presente
- `array_push($a, $x)`: aggiunge elementi in coda
- `array_pop($a)`: rimuove l'ultimo elemento
- `array_merge($a, $b)`: unisce due array

Funzioni per lavorare su ordine e struttura degli array:

- `array_keys($a)`: restituisce tutte le chiavi
- `array_values($a)`: restituisce tutti i valori
- `sort($a)`: ordina i valori in modo crescente
- `rsort($a)`: ordina i valori in modo decrescente
- `asort($a)`: ordina i valori mantenendo l'associazione chiave-valore
- `ksort($a)`: ordina le chiavi

- Come in JavaScript, la sintassi per definire una funzione è `function nome(argomenti) { ... }`
- Le funzioni possono prevedere un valore di ritorno da restituire tramite l'istruzione `return`
- Le variabili usate dentro a una funzione sono locali alla funzione stessa
- Di default, una funzione non vede le variabili globali (cioè create fuori dalla funzione stessa). Per poter accedere a una variabile globale, bisogna usare la dichiarazione `global` dentro la funzione
- Essendo debolmente tipato, PHP non richiede né la dichiarazione del tipo di dato restituito da una funzione, né il tipo dei parametri passati

Di seguito un file con estensione .php:

```
<html>
  <head>
    <title>Script Function PHP</title>
  </head>
  <body>
    <?php
      function somma($a,$b) {
        return $a + $b;
      }
      function printRandomValue() {
        $firstValue = 10;
        $secondValue = 20;
        echo "<h1>" . somma($firstValue,$secondValue) . "</h1>";
      }
      printRandomValue();
    ?>
  </body>
</html>
```

Questo è il corrispondente output HTML generato dall'interprete PHP:

```
<html>
  <head>
    <title>Script Function PHP</title>
  </head>
  <body>
    <h1>30</h1>
  </body>
</html>
```

- È possibile creare funzioni **variadiche**, ovvero passare alla funzione un numero arbitrario di argomenti “rimanenti”, usando la sintassi `...$args`
- È possibile passare alla funzione un **riferimento** come argomento, usando la sintassi `&$var_name`
- A partire da PHP 7 è possibile, inoltre, specificare per le funzioni sia il *tipo di dato* dei parametri di input che del valore di ritorno (la scelta demandata al programmatore). V. slide seguente.

Tipizzazione di input e output delle funzioni

- PHP 7 ha introdotto la possibilità di specificare i tipi dei parametri e del valore restituito da una funzione
- Questo rende il codice più chiaro e meno ambiguo

Esempio

```
declare(strict_types=1);  
  
function somma(int $a, int $b): int {  
    return $a + $b;  
}
```

- `strict_types=1` richiede che i tipi siano rispettati in modo rigoroso
- La verifica dei tipi aiuta a individuare errori prima e più facilmente

Funzioni anonime e arrow functions

- In PHP è possibile specificare funzioni anonime:

Esempio

```
$numeri = [1, 2, 3, 4];  
$doppi = array_map(function($n) {return $n * 2}, $numeri);
```

- In alternativa, PHP moderno supporta anche le arrow functions, utili per funzioni brevi (come quella nell'esempio)

Esempio

```
$numeri = [1, 2, 3, 4];  
$doppi = array_map(fn($n) => $n * 2, $numeri);
```

- Sono utili per operazioni semplici e immediate
- Rendono il codice più leggibile quando la logica è breve
- Il return è implicito

In PHP è possibile organizzare il codice usando classi e oggetti

- Una **classe** descrive una struttura e un comportamento
- Un **oggetto** è un'istanza concreta di una classe
- Le classi contengono **proprietà** e **metodi**
- Il costruttore `__construct()` inizializza l'oggetto
- `public`, `private` e `protected` controllano la visibilità di campi e metodi
- `extends` permette l'ereditarietà

Esempio di classe

```
class Persona {  
    public string $nome;  
    public function __construct(string $nome) {  
        $this->nome = $nome;  
    }  
    public function saluta(): string {  
        return "Ciao, sono " . $this->nome;  
    }  
}
```

- Scrivendo **new Persona("...")** verrà invocato il costruttore e istanziato un nuovo oggetto

- Un'eccezione rappresenta una situazione anomala che può essere intercettata e gestita
- PHP supporta il costrutto try-catch

Esempio

```
try {  
    $x = 10 / 0;  
} catch (Exception $e) {  
    echo "Errore: " . $e->getMessage();  
}
```

- Le eccezioni permettono di:
 - separare il codice normale dalla gestione degli errori
 - evitare interruzioni improvvise
 - mostrare messaggi più chiari all'utente

- 1 Introduzione
- 2 Note pratiche
- 3 Il linguaggio PHP
- 4 PHP come linguaggio lato server**
- 5 Connessione a un database tramite PHP

- `$GLOBALS` - contiene tutte le variabili globali (incluse le superglobali)
- `$_SERVER` - contiene informazioni sul server e sull'ambiente di esecuzione
- `$_GET` - contiene i parametri GET passati al server
- `$_POST` - contiene i parametri POST passati al server
- `$_FILES` - contiene i file passati al server
- `$_REQUEST` - è un riferimento a `$_GET`, `$_POST` e `$_FILES`
- `$_COOKIE` - contiene i cookie scambiati con il client
- `$_SESSION` - contiene le variabili di sessione
- `$_ENV` - contiene le variabili d'ambiente passate allo script

- Con `$_GET` e `$_POST` possiamo ispezionare i dati passati dal client
- Esempio:

- File `form.html`:

```
<form method="POST" action="saluto.php">  
    Nome: <input type="text" name="nome">  
    <input type="submit" value="Invia">  
</form>
```

- File `saluto.php`:

```
<?php  
if (isset($_POST["nome"])) {  
    echo "Ciao " . $_POST["nome"];  
}  
?>
```

- La variabile `$_SESSION` contiene tutti i dati di sessione
- Per memorizzare questi dati (in un certo file del server), la sessione dev'essere prima inizializzata tramite `session_start()`
- Ora tutto ciò che viene inserito nella variabile `$_SESSION` verrà conservato per le prossime esecuzioni dello script, fino allo scadere della sessione del client
- La variabile si può svuotare tramite `session_unset()`
- Chiamando `session_destroy()`, invece, viene anche cancellato il file relativo alla sessione (quindi sarà successivamente necessario chiamare di nuovo `session_start()`)

- La variabile `$_COOKIE` permette di leggere i cookie scambiati con il client
- È possibile chiedere al client di impostare un nuovo cookie tramite la funzione `setcookie()` (che prende in input la chiave e il valore del cookie, nonché eventuali opzioni)
- Esiste un cookie particolare avente come chiave `PHPSESSID`, ed è quello che permette alla sessione PHP di funzionare (viene impostato una volta chiamato `session_start()`)

include, require e versioni “once”

- PHP mette a disposizione alcuni costrutti per importare file esterni

Differenze principali

- `include 'file.php';` inserisce il file, ma genera solo un warning se il file non esiste
- `require 'file.php';` inserisce il file, ma genera un errore fatale se il file non esiste
- `include_once` e `require_once` evitano di includere più volte lo stesso file
- In pratica, `require_once` è molto usato per file di configurazione e librerie fondamentali

- In applicazioni reali, il codice PHP non dovrebbe essere scritto tutto in un unico file
- È buona pratica separare:
 - configurazione
 - logica applicativa
 - contenuto HTML
 - funzioni riutilizzabili
- Questo rende il codice:
 - più leggibile
 - più facile da mantenere
 - più semplice da riusare

Tramite `include` è possibile riutilizzare parti di pagina (es. header, footer, template)

L'idea è separare:

- logica PHP (dati)
- presentazione HTML (template)

A questo scopo, può risultare comoda la scorciatoia `<?= $var ?>` per stampare variabili in modo rapido

template.php

```
<h1>Ciao <?= $nome ?></h1>
<p>Benvenuto nel sito!</p>
```

index.php

```
<?php
$nome = "Luca";
include "template.php";
?>
```

Quando esegui index.php:

- viene incluso template.php
- la variabile \$nome è già disponibile
- <?= \$nome ?> viene sostituito con "Luca"

- 1 Introduzione
- 2 Note pratiche
- 3 Il linguaggio PHP
- 4 PHP come linguaggio lato server
- 5 Connessione a un database tramite PHP**

Per far sì che PHP possa utilizzare il DBMS PostgreSQL bisogna utilizzare l'estensione pgsql:

- Linux: Lanciare, tramite shell, il seguente comando: *sudo apt install php-pgsql*
- Nel file di configurazione `php.ini`, decommentare la riga `“; extension=pgsql”` (togliendo il `“;”` iniziale).
- Per chi usa XAMPP: solitamente il file `php.ini` è situato nella cartella `C:\xampp\php`

Tra le sue molteplici funzionalità, PHP fornisce API (funzioni di interfaccia) per accedere a database

Per connettersi con PHP ad un database, è necessario fornire

- **host**: nome dell'host
- **port**: numero di porto del server dove è attivo il DBMS
- **dbname**: nome del database
- **user**: nome dell'utente per la connessione al database
- **password**: relativa password

Un tipico esempio di connessione ad un database relazionale PostgreSQL è il seguente:

- `$dbconn = pg_connect("host=localhost, port=5433, dbname=EsempioConnessionePHP, user=postgres password=password") or die('Could not connect: ' . pg_last_error());`

Se la connessione va a buon fine (la funzione `pg_connect()` non restituisce alcun errore), allora potremo interagire con la base di dati “EsempioConnessionePHP” che si trova sulla nostra macchina (“localhost”) su un DBMS PostgreSQL che ascolta sul numero di porta “5433” con credenziali di accesso “postgres” e “password”. In caso contrario, la funzione `die()` terminerà lo script PHP con il messaggio di errore passatogli come parametro

Interazione con la base di dati

Una volta stabilita la connessione con la base di dati, sarà possibile interagire con essa per creare ed eliminare tabelle, fare operazioni di aggiunta/modifica/rimozione di tuple, così come poter eseguire query.

Continuando con l'esempio precedente, assumiamo che la base di dati "EsempioConnessionePHP" abbia una tabella di nome "organizzazione". Se siamo interessati a tutte le tuple di questa tabella, possiamo lanciare le seguenti istruzioni:

- `$query = 'SELECT * FROM organizzazione';`
- `$result = pg_query($query) or die('Query failed: ' . pg_last_error());`

La prima istruzione crea una variabile di nome `query` avente la query che siamo interessati a lanciare. La seconda istruzione lancia la query tramite la funzione `pg_query()` e salva il risultato nella variabile `$result`, che sarà un oggetto iterabile corrispondente ad un array per ogni tupla del risultato della query

Nella prossima slide mostreremo come maneggiare le tuple risultanti da una query grazie alla funzione `pg_fetch_array()` che consente di iterare su oggetti iterabili. Nello specifico l'obiettivo sarà quello di creare una tabella HTML dove ogni riga è una tupla della tabella "organizzazione"

Esempio completo

```
<html>
  <head>
    <title>Esempio in PHP</title>
  </head>
  <body>
    <?php
      $dbconn = pg_connect("host=localhost port=5433
        dbname=EsempioConnessionePHP user=postgres password=password")
        or die('Could not connect: ' . pg_last_error());
      $query = 'SELECT * FROM organizzazione';
      $result = pg_query($query) or die('Query failed: ' . pg_last_error());
      // Printing results in HTML
      echo "<table>\n";
      while ($line = pg_fetch_array($result, null, PGSQL_ASSOC)) {
        echo "\t<tr>\n";
        foreach ($line as $col_value) {
          echo "\t\t<td>$col_value</td>";
        }
        echo "\t</tr>\n";
      }
      echo "</table>\n";
      pg_free_result($result);
      pg_close($dbconn);
    ?>
  </body>
</html>
```

Alcune abitudini aiutano molto a scrivere codice più sicuro.

- Usare query parametrizzate invece di concatenare stringhe SQL (es. usando `pg_query_params()` al posto di `pg_query()`)
- Usare `password_hash()` e `password_verify()` per le password
- Fare l'escape dell'output HTML con `htmlspecialchars()`
- Impostare cookie con opzioni adeguate, ad esempio `HttpOnly` e `Secure`
- Controllare sempre i permessi dell'utente prima di eseguire azioni sensibili

PDO (PHP Data Objects) è un'interfaccia moderna per accedere ai database.

- Fornisce un modo uniforme per parlare con DBMS diversi
- Separa i dati dalla query tramite parametri
- È molto utile per ridurre il rischio di SQL injection

Rispetto alle funzioni specifiche del singolo DBMS, PDO rende il codice più portabile e più ordinato

- Durante la comunicazione con un database, possono verificarsi errori di diverso tipo:
 - connessioni fallite
 - query errate
 - ecc.
- `die()` interrompe immediatamente l'esecuzione dello script
- È utile nei casi semplici, ma in applicazioni più grandi conviene una gestione più strutturata
- A differenza dei metodi come `pg_connect()`, i PDO lanciano eccezioni che possono essere gestite in modo classico

PDO: connessione a PostgreSQL

```
$host = "localhost";
$port = "5432";
$db    = "nome_database";
$user  = "nome_utente";
$pass  = "password";
$dsn = "pgsql:host=$host;port=$port;dbname=$db;";
try {
    $pdo = new PDO($dsn, $user, $pass, [
        PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
    ]);
    echo "Connessione riuscita!";
} catch (PDOException $e) {
    echo "Errore di connessione: " . $e->getMessage();
}
```

```
$sql = "INSERT INTO utenti (nome, email)
        VALUES (:nome, :email)";
$stmt = $pdo->prepare($sql);

$stmt->execute([
    ':nome' => 'Mario Rossi',
    ':email' => 'mario@example.com'
]);
```

```
$sql = "SELECT id, nome, email FROM utenti";  
$stmt = $pdo->query($sql);  
  
$utenti = $stmt->fetchAll();  
  
foreach ($utenti as $utente) {  
    echo $utente['id'] . " - " .  
        $utente['nome'] . " - " .  
        $utente['email'] . "<br>";  
}
```


PDO: prepared statements in lettura

```
$sql = "SELECT * FROM utenti WHERE email = :email";  
$stmt = $pdo->prepare($sql);  
  
$stmt->execute([  
    ':email' => 'mario@example.com'  
]);  
  
$utente = $stmt->fetch();  
  
if ($utente) {  
    echo "Trovato: " . $utente['nome'];  
} else {  
    echo "Nessun utente trovato";  
}
```

- Documentazione ufficiale: <https://www.php.net/manual/>